

PROXMOX AUTOMATION SERIES · PHASE
3

The Agent Layer

Three agent roles land on every provisioned VM: Zabbix Agent 2 from the pinned 7.0 LTS repo, rsyslog forwarding everything over TCP with a disk-assisted queue, and licensed security installers modeled by stub scripts with a marker contract.

PVE 9.2 · TERRAFORM 1.15 · ZABBIX AGENT2 7.0 LTS
RSYSLOG OMFWD + DISK QUEUE · STUB MARKER CONTRACT
AUGUST 2026 · COMPANION REPO: PROXMOX-AUTOMATION

MONITOR · FORWARD · STUB-INSTALL

Contents

1. What Phase 3 Adds	1
2. Zabbix Agent 2, Pinned and Verified	5
3. Rsyslog Forwarding, the QRadar Pattern	9
4. The Stub Installer Pattern	12
5. Field Runbook and Real Transcripts	15
6. Troubleshooting the New Pieces	19

1. What Phase 3 Adds

This is the fourth module of the Proxmox Automation Series, and the first one built on top of a closed phase. Phase 2 ended with every storage claim field-verified: four serial-tagged disks converged into mounted XFS filesystems, a resize chain proven online, a reboot that handed everything back, and a growth run that rotated every device letter without the stack noticing. The machines exist, they hold data, and they come back. The question changes accordingly: not "does the machine exist" but "what is happening on it". That question is answered outside the guest, and the guest-side half of every answer is an agent.

Phase 3 installs three of them. A monitoring agent (Zabbix Agent 2) so a central server can poll the machine; a forwarding configuration (rsyslog) so the machine's logs leave it in the QRadar pattern; and two security agents (Trend Micro Deep Security, ManageEngine) that cannot be licensed in a homelab and therefore ship as stub installers behind a marker contract. All three arrive the same way the storage stack did: as numbered, self-contained, idempotent playbooks that read one `group_vars` section, converge the machine, and verify what they did before exiting green.

The original plan for this phase (chapter 1 of the series planning notes) named community roles, a Graylog container, and echo-installer stubs. What shipped differs in three places, each for a reason worth recording, and each deviation is a small lesson rather than a compromise. The table below maps plan to reality; the rest of this document explains why each row landed where it did.

The plan said	Phase 3 ships	Why the difference
<code>community.zabbix.zabbix_agent</code> role	hand-rolled <code>04_zabbix_agent.yml</code>	the repo pin is ours (7.0 LTS is the only current major publishing Ubuntu 26.04 packages), the control node installs nothing new, and the transcript stays visible. Swap path documented in chapter 2
a Zabbix server VM (docker, 4 GB)	an optional sidebar recipe	the deliverable is the agent; a one-LXC lab server is included as an optional step, and zabbix-get from ctrl-01 proves the last mile with no server at all
a Graylog container or rsyslog receiver	a socat one-liner on ctrl-01	the smallest thing that speaks TCP syslog; the QRadar pattern (forward everything, TCP, queued) survives unchanged, only the endpoint is a stub
<code>rsyslog *.* @receiver:514</code>	an omfwd action with a disk-assisted queue	the naive line loses messages whenever the receiver is down; the shipped rule buffers to disk and retries forever
stub shell scripts (echo + touch)	a marker contract with idempotency and a drift alarm	the plan's idea, upgraded: the harness is real, the payload is the only stubbed part

Table 1. The Phase 3 plan against what actually shipped.

The diff, file by file

File	Change	Purpose
ansible/playbooks/04_zabbix_agent.yml	new	Zabbix Agent 2 from the pinned official repo: key, sources line, package, config, handler, five-rung verification ladder
ansible/playbooks/05_rsyslog_forward.yml	new	rsyslog install plus the omfwd forwarding rule, config validation via rsyslogd -N1, and a tagged smoke test
ansible/playbooks/06_security_agents.yml	new	stub installers for both licensed agents, deployed as templates, run once behind a creates-guard, verified against the marker contract
ansible/playbooks/templates / (3 files)	new	zabbix_agent2.conf.j2, 50-qradar-forward.conf.j2, install.sh.j2: the series' first Jinja2 templates
ansible/inventory/group_vars/provisioned.yml	modified	a Phase 3 section: all agent knobs plus the security_agents list, the same single-source-of-truth pattern the storage layout uses
ansible/requirements.yml	comment only	Phase 3 is ansible.builtin end to end; the collection set on ctrl-01 does not change
terraform/	unchanged	agents are a guest-side concern; no disk, size or compute change is needed, and none was made
docs/phase-3-agents-guide.pdf	new	this document

Table 2. The complete Phase 3 diff: three playbooks, three templates, one group_vars section, and a deliberate no-op in terraform.

0

new collections on the control node. Phase 3 runs on ansible.builtin end to end; requirements.yml is untouched, and the sandbox validation covers exactly what the field will run.

Scope and boundaries

As always in this series, the additions are scoped so each can be verified alone. The three playbooks share one group_vars section but do not depend on each other; any of them can run first, re-run, or be skipped entirely. The boundaries below are the honest ones, and the biggest one deserves emphasis: this phase delivers and verifies the AGENT SIDE. A production Zabbix server, a production QRadar, and licensed installers are all outside it, and each has a documented stand-in.

In scope for Phase 3	Deliberately out of scope
Zabbix Agent 2 installed, configured and verified from the pinned 7.0 LTS repo	a production Zabbix server deployment (an optional lab recipe appears as a sidebar)
rsyslog forwarding everything over TCP with a disk-assisted queue	QRadar itself, TLS transport, and log routing rules beyond everything
stub installer harness with a marker contract for both licensed agents	the licensed installers (stubbed by design until they exist)
a lab receiver (one socat line) and the receipts it produces	continuous monitoring semantics: templates, triggers, dashboards
a field-run runbook: every step green on 28 August 2026, one accidental outage included	Semaphore UI, dynamic inventory, form-driven pipelines (Phase 4)

Table 3. What Phase 3 builds, and what it leaves alone.



Figure 1. Phase 3 in one picture: one group_vars section feeds three playbooks; three agents land on the guest; ctrl-01 stands in for both central systems.

2. Zabbix Agent 2, Pinned and Verified

The monitoring agent is Zabbix Agent 2, and the choice is not sentimental. The classic agent forks a process per metric; Agent 2 is a Go rewrite with a plugin architecture, a persistent connection for active checks, and the same 10050 port and config vocabulary the ecosystem already knows. For a fleet of cloned DB servers it is the default answer, and the knobs a server-side host entry expects (Server, ServerActive, Hostname) are exactly the knobs this playbook templates.

Why not the `community.zabbix` role (yet)

The original plan said to use the `community.zabbix.zabbix_agent` role, and that is a fine plan at fleet scale. For this phase the playbook is hand-rolled instead, for three field reasons. First, the repository pin is ours to control: 7.0 LTS is, as of August 2026, the only current Zabbix major that publishes packages for Ubuntu 26.04, and that fact belongs in a variable we set deliberately rather than inside a role's defaults. Second, the control node installs nothing new: Phase 3 stays `ansible.builtin`-only, so `requirements.yml` and the sandbox validation keep describing reality. Third, this series shows every command it runs; a role wrapping forty tasks would replace the transcript with a parameter list. When a second OS family or a parameter sprawl arrives, the swap is an evening: the `group_vars` section already speaks the role's vocabulary.

The repository pin, checked live

Choosing the major version is a research step, not a guess, and the research is repeatable: a `HEAD` request against the repo layout answers it. The check was run while writing this guide, with the results below, and the same check will answer the question again when Ubuntu 26.04's successor ships or Zabbix 8.x catches up.

THE LIVE CHECK, AND WHAT IT SAID (AUGUST 2026)

```
$ curl -sI https://repo.zabbix.com/zabbix/7.0/ubuntu/dists/resolute/Release | head -1
HTTP/2 200 # 7.0 LTS publishes for Ubuntu 26.04 (codename resolute)
$ curl -sI https://repo.zabbix.com/zabbix/8.0/ubuntu/dists/resolute/Release | head -1
HTTP/2 404 # 8.0 does not (yet)
$ curl -sI https://repo.zabbix.com/zabbix/8.2/ubuntu/dists/resolute/Release | head -1
HTTP/2 404 # 8.2 does not (yet)
```

Source	Version for Ubuntu 26.04	Notes
repo.zabbix.com, major 7.0 (pinned)	zabbix-agent2 1:7.0.30-1+ubuntu26.04	the choice: current LTS train, newest patch, official support
repo.zabbix.com, majors 8.0 / 8.2	not published	no resolute section as of August 2026; re-check before switching majors
Ubuntu universe (no repo needed)	zabbix-agent2 1:7.0.22+dfsg-1	older patch of the same train; the fallback if the official repo ever drops this release

Table 4. Where zabbix-agent2 can come from for this guest, live-checked while writing this guide.

One subtlety hides in that table: universe also ships zabbix-agent2, so the installed version is an ELECTION apt holds between two repositories, and apt votes for the higher version number. Today that elects 7.0.30 from repo.zabbix.com over 7.0.22 from universe, which is what we want; but an election is not a guarantee, and a playbook that pins a repo without reading the result back is just hoping loudly. So 04 reads `apt policy zabbix-agent2` back every run and asserts three things: something is installed, the candidate names repo.zabbix.com, and the version carries the pinned major. If universe ever wins that election, the run fails before anything else happens.

The install, key first

THE REPOSITORY TASKS IN 04_ZABBIX_AGENT.YML (TRIMMED)

```
- name: Create the apt keyrings directory (modern signed-by layout)
  ansible.builtin.file:
    path: /etc/apt/keyrings
    state: directory
    mode: "0755"

- name: Install the Zabbix official repo signing key
  ansible.builtin.get_url:
    url: https://repo.zabbix.com/zabbix-official-repo.key
    dest: /etc/apt/keyrings/zabbix-official-repo.asc
    mode: "0644"

- name: Add the pinned Zabbix apt repository (major version knob)
  ansible.builtin.apt_repository:
    repo: >-
    deb [signed-by=/etc/apt/keyrings/zabbix-official-repo.asc]
    https://repo.zabbix.com/zabbix/{{ zabbix_agent_version }}/ubuntu
    {{ ansible_facts['distribution_release'] }} main
    filename: zabbix-official-repo
    state: present

- name: Install Zabbix Agent 2
  ansible.builtin.apt:
    name: zabbix-agent2
    state: present
```


The order matters twice over. The key lands before the sources line that references it, because the `apt_repository` task runs `apt update` as soon as the line changes, and an update against a missing key fails the whole play. And the release name comes from the facts, not from a hardcoded "resolute", so the same playbook survives the next Ubuntu upgrade; only the major version knob is pinned by hand.

The configuration, and the series' first handlers

The config is a template, deliberately close to the packaged default: only the lines a server-side host entry cares about are variable, and TLS plus `HostInterface` belong to a production-hardening pass, not a lab. The template notifies a handler, the first handler in this series, and there is a reason it exists at all: without one, a config change would either need a blind restart task that runs every time, or no restart at all. With one, the agent restarts exactly when its config changed. A `flush_handlers` call then runs pending restarts BEFORE the verification tasks, so what gets verified is the running agent, not a file waiting for a restart nobody scheduled.

TEMPLATES/ZABBIX_AGENT2.CONF.J2 (THE WHOLE FILE)

```
# {{ ansible_managed }}
# Phase 3 (04_zabbix_agent.yml). Deliberately close to the packaged
# default: only the lines a server-side host entry cares about are
# variable. TLS and HostInterface belong to the production-hardening
# pass, not the lab.
PidFile=/run/zabbix/zabbix_agent2.pid
LogFile=/var/log/zabbix/zabbix_agent2.log
Server={{ zabbix_agent_server }}
ListenPort={{ zabbix_agent_listenport }}
ServerActive={{ zabbix_agent_serveractive }}
Hostname={{ zabbix_agent_hostname }}
Include=/etc/zabbix/zabbix_agent2.d/plugins.d/*.conf
```

Knob (group_vars)	Lands as	Lab value
<code>zabbix_agent_version</code>	the pinned repo line	7.0 (LTS)
<code>zabbix_agent_server</code>	<code>Server=</code> (who may poll this agent)	192.168.122.10 (ctrl-01)
<code>zabbix_agent_serveractive</code>	<code>ServerActive=</code> (where active checks go)	192.168.122.10 (ctrl-01)
<code>zabbix_agent_listenport</code>	<code>ListenPort=</code>	10050
<code>zabbix_agent_hostname</code>	<code>Hostname=</code> (must match the host entry)	vm-test-01, via <code>inventory_hostname</code>

Table 5. The five knobs, where they land, and what the lab sets them to.

The verification ladder

Every run of 04 ends with the same five rungs, weakest proof first, none of which needs a Zabbix server to exist. That is the point: the agent-side contract is complete without the server, and the server, when it arrives, only adds the last mile. The rungs:

- **Election.** `apt policy zabbix-agent2` read back: installed, from `repo.zabbix.com`, at the pinned major.
- **Service.** `service_facts` says `zabbix-agent2` is running.
- **Port.** `wait_for` connects to `127.0.0.1:10050` and does not time out.
- **Local metric.** `zabbix_agent2 -t agent.version` parses the config, loads the plugins and answers; the assert demands `rc=0` and the item key in the output, not an exact format.
- **Config.** the deployed file is slurped back and must contain the exact `Server`, `ServerActive` and `Hostname` lines the knobs say.

The cheapest last-mile proof, when you want one without building anything, runs from `ctrl-01`: install `zabbix-get` there and ask the agent directly. The agent already trusts `ctrl-01` (it is the `Server=` value), so this polls the real path a server would poll, with zero servers deployed. The field run of 28 August added a packaging surprise worth recording: Ubuntu 26.04's universe carries no `zabbix-get` package at all, so the plain install fails with `Unable to locate package`, while the pinned official repo publishes it at the same version as the agent. The light path is therefore to give `ctrl-01` the same repository 04 gives the guest, and only then install the package. The heavy path, taken live during the field run before anyone checked, was discovering that `zabbix-server-pgsql` also ships the binary, at the price of a whole PostgreSQL and 241 MB of collateral on the control node; the receipt below is identical either way.

THE LAST MILE, NO SERVER REQUIRED (RUN ON CTRL-01)

```
# light path: ctrl-01 needs the official 7.0 repo first (universe
# has no zabbix-get for 26.04); then simply:
sudo apt-get install -y zabbix-get
zabbix_get -s 192.168.122.50 -k agent.version

# field receipt, 28 August 2026 (the binary arrived the heavy way,
# inside zabbix-server-pgsql; the answer is the same either way):
# 7.0.30
```

Optional: a lab Zabbix server in one LXC

When graphs are wanted, the lab server is one container away, and universe already carries the whole 7.0 train, so no extra repository is needed on the server side. The recipe below reuses the LXC pattern `ctrl-01` itself uses. One wiring note before it: the agent accepts polls from `zabbix_agent_server` only, so either add the container's IP to that knob as a comma-separated list, or keep polling from `ctrl-01` as above.

OPTIONAL LAB SERVER (ON A PVE NODE, THEN INSIDE THE CONTAINER)

```
# pick the exact template name first: pveam list nfs-shared
pct create 901 nfs-shared:vztmpl/ubuntu-26.04-standard_amd64.tar.zst \
--hostname zbx-01 --cores 2 --memory 2048 --rootfs nfs-shared:8 \
--net0 name=eth0,bridge=vbr0,ip=192.168.122.60/24,gw=192.168.122.1 \
--unprivileged 1
pct start 901 && pct enter 901

# inside: universe has the whole 7.0 set; debconf wires the database
apt-get update && apt-get install -y postgresql zabbix-server-pgsql \
zabbix-frontend-php php-pgsql
systemctl restart zabbix-server apache2
# browse http://192.168.122.60/zabbix (Admin / zabbix), add host
# vm-test-01 at 192.168.122.50; the agent is already verified
```

3. Rsyslog Forwarding, the QRadar Pattern

A DB server that logs only to its own disk keeps its evidence next to the incident. The production answer is central collection, in this environment QRadar, and the guest-side half of that answer is always the same shape: rsyslog, forwarding everything, over TCP, with a queue. The original plan tested the pattern with a one-liner, and the one-liner is exactly where everyone starts, so it is worth seeing why it is also where everyone gets burned.

THE NAIVE RULE, AND WHAT SHIPS INSTEAD

```
# what everyone writes first (and what loses messages):
*. * @@192.168.122.10:5514

# what 05_rsyslog_forward.yml deploys instead:
action(type="omfwd"
target="192.168.122.10"
port="5514"
protocol="tcp"
queue.type="LinkedList"
queue.filename="qradar_fwd"
queue.maxDiskSpace="100m"
queue.saveOnShutdown="on"
action.resumeRetryCount="-1")
```

The naive rule works on the happy path and loses messages on every other one. While the receiver is down, for maintenance, network or restart, plain TCP forwarding either blocks local logging or drops lines, depending on rsyslog's mood and defaults; UDP drops silently always. The shipped action is the production shape instead: TCP because QRadar's ingest expects reliable transport, a LinkedList queue that is fast and in-memory while the receiver answers, `queue.filename` turning it into a disk-assisted queue that spills to `/var/spool/rsyslog` when it does not, `saveOnShutdown=on` so an rsyslog restart keeps the queue intact, and `action.resumeRetryCount="-1"` so the rule retries forever

instead of quietly disabling itself after a few failures. A hundred megabytes of buffer is a lab number; the knob exists so production can size it to an SLA.

Knob (group_vars)	Meaning	Lab value	Production (QRadar)
rsyslog_receiver_host	where everything goes	192.168.122.10 (ctrl-01)	the collector IP
rsyslog_receiver_port	TCP port	5514, unprivileged	514
rsyslog_forward_protocol	transport	tcp	tcp (TLS later)
rsyslog_queue_maxdisk	on-disk buffer cap	100m	sized to the SLA
rsyslog_test_tag	smoke-test tag	phase3-test	unchanged

Table 6. Five knobs carry the whole lab-to-production distance; the rule's structure never changes.

Why port 5514 in the lab: binding 514 needs root, and the lab receiver is a one-liner rather than a service. An unprivileged port keeps the stub honest (no root daemon on the control node) while preserving the pattern; the production switch is one variable. The receiver itself is deliberately dumb, because anything that appends lines to a file proves the path. Its shape changed once, for a field reason: edition 1 ran it in the foreground, the field run proved that a foreground one-liner dies with the SSH session that started it, and edition 2 runs it under nohup so the receiver survives the console that launched it:

THE LAB RECEIVER, SESSION-PROOF (RUN ON CTRL-01)

```
sudo apt-get install -y socat
nohup socat -u TCP-LISTEN:5514,reuseaddr,fork \
OPEN:/tmp/qradar-stub.log,creat,append >/dev/null 2>&1 &

# two behaviors worth knowing before they surprise you:
# the file is created on the FIRST inbound connection, not at startup
# after an outage, rsyslog retries a suspended action every ~30 s

# a real line from the field run (RFC3164 header, hostname, tag):
# Aug 28 09:33:18 vm-test-01 phase3-test: phase 3 forwarding smoke message
```

Inside the guest, the playbook does the rest, and the task list is short because rsyslog is small. Edition 1 claimed the Ubuntu cloud image ships without rsyslog, the same minimal-image lesson the storage phase learned with lvm2 and xfsprogs; the field run proved that claim wrong, and the correction deserves its own sentences. dpkg's log on the guest has no install line for rsyslog at all, only a first-boot trigger pass (trigproc, then half-configured, then installed, all at 2026-08-26 16:16:19), which is the signature of a package the image carried along and configured on first boot. The package even ships the spool directory, and its postinst hardens it to mode 0700, owner syslog:adm; the playbook's spool task declares 0755, so the first run's change in that task was a mode normalization, not a directory creation, and the syslog:adm ownership in the receipts is the package's fingerprint, not Ansible's. The install task stays anyway,

because a guard that costs nothing on every later run is exactly what idempotency is for; only its name and comment changed in edition 2. Then the rule lands as a template, a handler restarts the daemon (flushed before verification, same pattern as chapter 2), and the proof runs in two halves: `rsyslogd -N1` parses the full configuration without running it, the cheapest possible "the daemon will actually start" check, and a tagged smoke message goes in via `logger` and must come back out in the local log, proving rsyslog really processed it. The receiver-side line is the field receipt, chapter 5.

The outage that ran itself

The field run of 28 August 2026 contained twelve minutes this guide could not have scheduled better. The receiver had been started as a plain foreground one-liner in an SSH session; when that session closed, socat died with it, silently, and nobody noticed because nothing was forwarding yet. Then the runbook ran: 05 deployed the rule, restarted rsyslog, and went green, its smoke test proving the local half while the real destination was dead. A second tagged message, sent on purpose while the receiver was down (its text says exactly that, in Turkish: queue exam, sent while the receiver was down), joined the first in the queue. This is the exact scenario the `omfwd` action was shaped for, arriving by accident instead of by plan, and every queue claim in the block above got its receipt:

THE OUTAGE AND THE DRAIN, AS THEY ACTUALLY HAPPENED (28 AUGUST 2026)

```
# receiver gone with its session; nothing is listening:
$ ss -tlnp | grep 5514
(no output)

# the guest keeps every line; restarting rsyslog while the receiver
# is down proves saveOnShutdown=on across a daemon restart:
$ ssh ubuntu@192.168.122.50 'sudo systemctl restart rsyslog && ls -la
/var/spool/rsyslog/'
-rw----- 1 syslog syslog 193147 Aug 28 09:45 qradar_fwd.00000001
-rw----- 1 syslog syslog 567 Aug 28 09:45 qradar_fwd.qi

# the receiver returns, session-proof this time:
$ nohup socat -u TCP-LISTEN:5514,reuseaddr,fork \
OPEN:/tmp/qradar-stub.log,creat,append >/dev/null 2>&1 &
[1] 3874

# one resume cycle later (~30 s), the backlog arrives at once:
$ wc -l /tmp/qradar-stub.log
942 /tmp/qradar-stub.log
$ grep kuyruk /tmp/qradar-stub.log
<13>Aug 28 09:34:21 vm-test-01 phase3-test: kuyruk sinavi: alici kapaliyken gonderildi

# and the wire, seen from the guest:
$ ssh ubuntu@192.168.122.50 'ss -tn | grep 5514'
ESTAB 0 0 192.168.122.50:37010 192.168.122.10:5514
```

Read the timestamps closely, because they are the whole point. The exam line was generated at 09:34:21 UTC; the receiver file came into being only when the revived socat accepted its first connection, roughly twelve minutes later. The line is in the file anyway, wearing its ORIGINAL generation stamp, because it

spent those minutes in the disk queue. Syslog lines carry when they were made, not when they were delivered; a receiver full of lines stamped in the past is not a clock problem, it is late delivery working. The naive rule from the top of this chapter would have none of this: both smoke messages, the exam message and roughly 940 more lines of session and Ansible noise (the rule forwards everything, so a twelve-minute backlog is big by design) would simply be gone. One more honest observation: the queue file lingered at about 193 KB after the drain, 553 bytes larger than before it, and with the receiver back the live traffic flows through memory while the disk half sleeps until the next outage; rsyslog truncates the segment on its own schedule, so the receipts on the receiver side, not the file size, are the evidence that the drain happened.

One honest limitation remains, restated after the field run: the outage above proved the queue under a twelve-minute receiver loss and a daemon restart, which is the shape of most real incidents, but not its ceiling. Nothing here exercises the 100 MB cap under a sustained outage, TLS transport, or a receiver that accepts connections and then stalls. Those belong to the chaos chapter the series has not needed yet, and the knobs are already in place for the day it does.

4. The Stub Installer Pattern

The licensed security tools cannot run here: no installer binaries, no licenses, no call-home infrastructure, and pretending otherwise would be the one dishonest move this series refuses to make. But the PIPELINE that will deploy them can run today, and that is what 06_security_agents.yml builds. The original plan said it plainly: stub shell scripts, echo instead of install, a marker file to prove it happened, end-to-end testable now, real installers swapped in the day the licenses arrive. The plan's sentence survives as the design.

What the plan did not spell out, and what turns an echo into an engineering pattern, is the interface. The playbook must never care whether the payload is a stub or a licensed binary, so there is exactly one stable contract between them: the marker file. Every installer, stub or licensed, ends by writing `<install_dir>/.installed` with five fields, and the playbook only ever talks to that file. It deploys a script, runs it once, and reads the marker back; whatever the script did in between is the script's business.

TEMPLATES/INSTALL.SH.J2 (TRIMMED TO THE CONTRACT)

```
#!/bin/sh
# {{ item.display_name }} - installer ({{ item.mode }} mode).
# THE MARKER CONTRACT (the only stable interface): whatever this
# script is, it ends by writing {{ item.install_dir }}/installed.
set -eu
INSTALL_DIR="{{ item.install_dir }}"
MARKER="{{ item.install_dir }}/installed"

echo "[{{ item.name }}] installer starting: version {{ item.version }}..."

# ---- installer block -----
# STUB: simulates the licensed installer. Replace ONLY this block
# (download, silent flags, service start) when the license arrives.
echo "[{{ item.name }}] stub payload: no binaries deployed, no license used"
# ---- end installer block -----

mkdir -p "$INSTALL_DIR"
{
echo "agent={{ item.name }}"
echo "version={{ item.version }}"
echo "mode={{ item.mode }}"
echo "installed_at=$(date -Is)"
echo "host=$(hostname -s)"
} > "$MARKER"

echo "[{{ item.name }}] marker written: $MARKER"
```

Marker field	Who writes it	Who reads it, and against what
agent=name	the installer script	06 asserts it against the loop item
version=...	the installer script	06 asserts it against group_vars; a mismatch is the drift alarm
mode=stub real	the installer script	06's guard only admits stub today; real unlocks with the licensed swap
installed_at=ISO date	the installer script	forensics: when this machine last converged
host=short name	the installer script	forensics: proves the marker was not copied in

Table 7. The marker contract: five lines, the whole interface.

Idempotency, drift, and the alarm

The installer runs exactly once per agent, guarded the Ansible-native way: the command task carries `creates: <install_dir>/installed`, so a re-run finds the marker and skips the execution. Unlike a `when:` condition, the `creates-guard` reports as `ok` rather than `skipped`, which keeps the re-run recap clean: this was verified in the sandbox, where the second run of the real playbook file landed on

ok=8 changed=0 with nothing skipped. The arithmetic of the first run is equally plain: directories, scripts and installer executions are the only things that change, so ok=8 changed=3.

Bump a version in `group_vars` and the behaviour changes on purpose: the script template re-renders with the new version, but the marker on disk still records the old one, and the marker assert FAILS, naming the agent and the expected version. That is the drift alarm. A new desired version sitting next to an old marker is exactly the situation a fleet manager wants flagged, not auto-repaired, because the repair (run the installer again) is a decision, not a default. The documented force-reinstall is one ad-hoc command, and the sandbox exercised the whole arc: first run, clean re-run, version bump failing loudly, marker removal, reinstall.

THE DOCUMENTED FORCE-REINSTALL (RUN FROM ANSIBLE/)

```
ansible provisioned -m file \  
-a "path=/opt/TrendMicro/.installed state=absent" -b \  
ansible-playbook playbooks/06_security_agents.yml
```

The swap-in runbook, for the day the licenses arrive

- Replace the installer block inside `templates/install.sh.j2`: download the licensed binary, run it with its silent flags, start its service. Leave the marker block byte-for-byte as it is.
- Move the real payload's facts into the loop entry in `group_vars`: keep the name and `install_dir`, bump the version to the real one, flip mode to real.
- Flip the guard: 06's `pre_task` currently refuses mode real with a message pointing at this chapter; admitting it is a one-line change made consciously, not by accident.
- Re-run. The playbook cannot tell the difference: it deploys a script, runs it once, reads the marker back.

marker

is the interface. The playbook never inspects the payload; the payload never bypasses the marker. Everything else about either side is free to change.

5. Field Runbook and Real Transcripts

The field run happened on 28 August 2026, and this chapter is edition 2 of itself: every transcript below is real, trimmed only for width. Two conventions carry over from edition 1. Predictions that missed stay on the page and get explained, never quietly overwritten; and the counts still come with their arithmetic, so a re-run can be read instead of trusted. The score at the end: one prediction missed by exactly two (04's changed count, for one repeated reason), one arrived twelve minutes late and better than written (the receiver receipt, through the disk queue, chapter 3), and everything else landed exactly as computed.

Step	Command (from ansible/ on ctrl-01)	Recap	Status
0	receiver, session-proof: nohup socat -u TCP-LISTEN:5514,reuseaddr,fork OPEN:/tmp/qradar-stub.log,creat,append &	no output; the file appears on first connection	field
1	ansible-playbook playbooks/04_zabbix_agent.yml	ok=19 changed=5 (edition 1 said 7)	field
1b	same again	ok=18 changed=0, run twice	field
2	ansible-playbook playbooks/05_rsyslog_forward.yml	ok=13 changed=3	field
2b	grep phase3-test /tmp/qradar-stub.log (on ctrl-01)	seven lines, twelve minutes late, via the queue	field
3	ansible-playbook playbooks/06_security_agents.yml	ok=8 changed=3	field
3b	same again	ok=8 changed=0	field
4	zabbix_get -s 192.168.122.50 -k agent.version (on ctrl-01)	7.0.30	field
5	the queue exam: receiver down, message sent, receiver revived	942 lines delivered, original timestamps intact	field

Table 8. The run at a glance, 28 August 2026. Every row ran; the one number edition 1 got wrong is row 1, explained below.

Step 1: the monitoring agent

04_ZABBIX_AGENT.YML, FIRST RUN, 28 AUGUST 2026 (TRIMMED)

```

TASK [Create the apt keyrings directory (modern signed-by layout)] ***
ok: [vm-test-01] # the resolute image already ships /etc/apt/keyrings

TASK [Install the Zabbix official repo signing key] *****
changed: [vm-test-01]

TASK [Add the pinned Zabbix apt repository (major version knob)] *****
changed: [vm-test-01]

TASK [Install Zabbix Agent 2] *****
changed: [vm-test-01] # and apt's postinst starts+enables the service

TASK [Deploy the agent configuration] *****
changed: [vm-test-01]

TASK [Enable and start Zabbix Agent 2] *****
ok: [vm-test-01]

RUNNING HANDLER [Restart Zabbix Agent 2] *****
changed: [vm-test-01]

TASK [Receipt - agent on duty] *****
ok: [vm-test-01] => {
  "msg": "zabbix-agent2 (repo 7.0) listening on port 10050; passive checks
accepted from 192.168.122.10, active checks sent to 192.168.122.10 as
host \"vm-test-01\". Self-test: agent.version [s|7.0.30]"
}

PLAY RECAP *****
vm-test-01 : ok=19 changed=5 unreachable=0 failed=0

```

Edition 1 predicted changed=7 and the field returned changed=5, and both missing changes teach the same lesson about assuming package state. The keyrings directory landed ok because Ubuntu 26.04's cloud image already ships /etc/apt/keyrings, so edition 1's minimal-image assumption, inherited from the lvm2 and xfsprogs story of the storage phase, was wrong here. The service task landed ok because apt's postinst had already started and enabled zabbix-agent2 before the systemd task ran, the same postinst lesson 05 was about to teach again. Everything the playbook ASSERTS still ran and still passed: the election, the service state, the port, the local metric, the config readback. The re-run held exactly: ok=18 changed=0, run twice, the 19-to-18 drop being the handler that no longer runs because nothing notified it.

Step 2: the forwarding

05_RSYSLOG_FORWARD.YML, FIRST RUN, 28 AUGUST 2026 (TRIMMED)

```

TASK [Install rsyslog (Ubuntu cloud images ship without it)] *****
ok: [vm-test-01] # edition 1's task name; chapter 3 has the field
# correction (the image ships it, configured at boot)

TASK [Ensure the queue spool directory exists] *****
changed: [vm-test-01] # the postinst's 0700 normalized to 0755

TASK [Deploy the forwarding rule] *****
changed: [vm-test-01]

TASK [Enable and start rsyslog] *****
ok: [vm-test-01] # already active since first boot configured it

RUNNING HANDLER [Restart rsyslog] *****
changed: [vm-test-01]

TASK [Receipt - forwarding is live] *****
ok: [vm-test-01] => {
  "msg": "rsyslog forwards *.* to 192.168.122.10:5514 over TCP with a
  disk-assisted queue (100m at /var/spool/rsyslog). ..."
}

PLAY RECAP *****
vm-test-01 : ok=13 changed=3 unreachable=0 failed=0

```

Three changed, as predicted, but not the three edition 1 guessed: the spool directory mode, the rule template and the handler restart, with the package task landing ok because rsyslog had been on the guest since first boot (chapter 3 tells that story with the dpkg receipts). The re-run held exactly: ok=12 changed=0. One quirk in the receipt deserves a sentence, because it will save a reader an hour: the Local echo line shows the GREP task's own invocation, not the smoke message, because Ansible logs every command it runs to syslog, that logged line contains the literal string phase3-test, and the grep therefore finds itself. The assert behind it demanded the real smoke text, which was present, so the recursion is cosmetic; it is also a preview of why the receiver file grows so fast, below.

THE RECEIVER-SIDE RECEIPT, 28 AUGUST 2026 (ON CTRL-01)

```

$ grep phase3-test /tmp/qradar-stub.log
<13>Aug 28 09:33:18 vm-test-01 phase3-test: phase 3 forwarding smoke message
<13>Aug 28 09:33:39 vm-test-01 phase3-test: phase 3 forwarding smoke message
<13>Aug 28 09:34:21 vm-test-01 phase3-test: kuyruk sinavi: alici kapaliyken gonderildi

# wc -l /tmp/qradar-stub.log
# 942 /tmp/qradar-stub.log <- normal: the rule forwards everything

```

These three lines are the receipt the whole pattern exists for, and they arrived twelve minutes after they were generated, because the receiver was dead the whole time and everything came through the disk queue once it was revived (the full story, with the queue surviving a daemon restart, is chapter 3's outage section). Four more lines matched the same grep and are not shown: the Ansible invocations of the two logger and two grep tasks, whose command lines contain the tag, forwarded like everything else. That is

also why wc -l says 942: a twelve-minute *.* backlog of session noise and Ansible's per-command logging is big by design, and the number is evidence, not a problem.

Step 3: the stub agents

06_SECURITY_AGENTS.YML, FIRST RUN, PLUS BOTH MARKERS

```
TASK [Run each installer once (the marker is the creates-guard)] *****
changed: [vm-test-01] => (item=trendmicro)
changed: [vm-test-01] => (item=manageengine)

PLAY RECAP *****
vm-test-01 : ok=8 changed=3 unreachable=0 failed=0

# both markers, read back from the guest (28 August 2026):
$ ssh ubuntu@192.168.122.50 'cat /opt/TrendMicro/.installed
/opt/ManageEngine/.installed'
agent=trendmicro
version=1.0.0-stub
mode=stub
installed_at=2026-08-28T09:12:01+00:00
host=vm-test-01
agent=manageengine
version=1.0.0-stub
mode=stub
installed_at=2026-08-28T09:12:02+00:00
host=vm-test-01
```

This is the one playbook whose counts were already proven before the field: the sandbox had run the shipped file through first run, clean re-run, version bump and forced reinstall, predicting ok=8 changed=3 then ok=8 changed=0. The field matched both exactly, which makes 06 the series' first playbook with its numbers proven twice over. The marker timestamps sit one second apart, two loop iterations of the same task; the drift alarm and the swap-in runbook of chapter 4 remain exactly as written.

The prediction scorecard

Prediction (edition 1)	Field outcome (28 August 2026)	Verdict
04 first run ok=19 changed=7	ok=19 changed=5	missed by two: image ships /etc/apt/keyrings, postinst starts the service
04 re-run ok=18 changed=0	ok=18 changed=0, twice	exact
05 first run ok=13 changed=3	ok=13 changed=3, but the changed three are spool mode, template, handler (not the package)	exact count, corrected composition
05 re-run ok=12 changed=0	ok=12 changed=0	exact
06 first run ok=8 changed=3, re-run ok=8 changed=0	both exact	exact, twice-proven (sandbox and field)
a phase3-test line right after 05	seven lines, twelve minutes late, through the disk queue	arrived better than written
zabbix_get answers the version	7.0.30, but zabbix-get is not in universe (chapter 2)	answer exact, path corrected

Table 9. What edition 1 promised against what the field delivered. One miss, one upgrade, five exact.

6. Troubleshooting the New Pieces

The storage chapters built a troubleshooting table from field failures; this one started, in edition 1, from predicted failures instead. The field run of 28 August has now been through all of it, so the table below mixes rows the field proved with rows it merely left standing, and the field contributed four new rows nobody predicted, including the best one: a receiver that died with its SSH session and taught the guide its nohup lesson.

Symptom	Cause	Fix
apt update fails 404 on repo.zabbix.com .../dists/resolute/Release after adding the repo	Zabbix has not published this Ubuntu release for the pinned major (8.0 and 8.2 lack resolute as of August 2026)	live-check with the curl -sI command from chapter 2; keep zabbix_agent_version on a major that publishes for your release
the election assert fails: candidate comes from archive.ubuntu.com (7.0.22)	the official repo lost the apt election: key missing or sources line wrong	check /etc/apt/keyrings/zabbix-official-repo.asc and /etc/apt/sources.list.d/zabbix-official-repo.list, run apt update, re-run the play
apt tasks fail with lock errors	unattended-upgrades is holding the dpkg lock	wait a few minutes and re-run; the play is idempotent and resumes cleanly
agent is active but a server's polls time out	Server= does not list the polling host; the agent silently drops untrusted polls	add the server IP to zabbix_agent_server (comma-separated list) and re-run; or poll from ctrl-01 with zabbix-get
wait_for on 10050 times out although the service shows active	a config error kept the listener down while systemd still reports the unit running	journalctl -u zabbix-agent2 -n 50; zabbix_agent2 -t agent.version prints the raw parse error
rsyslogd -N1 fails mentioning the queue and a work directory	the base /etc/rsyslog.conf lacks global(workDirectory=...); queue.filename needs it	set it to /var/spool/rsyslog, or drop queue.filename; Ubuntu's packaged conf ships it
05 is green but /tmp/qradar-stub.log stays empty	socat not running (field-proven cause: it died with the SSH session that started it), wrong port, or a TCP listener against the udp knob	check the local /var/log/syslog line first (proves the guest side), then ss -tlnp grep 5514, then the protocol; restart with the nohup form
the receiver file never appears although socat is LISTENing	socat creates the OPEN: file on the first inbound connection, and rsyslog retries a suspended action on its own ~30 s cycle	wait a minute and re-check; ss -tn grep 5514 on the guest proves the wire, journalctl -u rsyslog -n 20 diagnoses the sender
the receiver works, then dies some time later	a foreground one-liner dies with the SSH session that spawned it (SIGHUP)	run it session-proof: nohup socat ... >/dev/null 2>&1 & (the field run learned this the honest way, chapter 3)
zabbix-get fails to install on ctrl-01: Unable to locate package	Ubuntu 26.04 universe carries no zabbix-get; only the official Zabbix repo publishes it	give ctrl-01 the 7.0 repo (chapter 2) and install from there, or accept the heavy path: the binary also ships inside zabbix-server-pgsql
wc -l on the receiver file shows hundreds of lines right after an outage	the rule forwards everything, so a backlog contains every syslog line generated while the receiver was down, including Ansible command logging	normal and by design; grep for the tag, do not count lines (field receipt: 942)

Symptom	Cause	Fix
the marker assert fails after a version bump	the drift alarm doing its job: old marker, new desired version	intended; force the reinstall with the chapter 4 command, or roll the version back
the guard rejects mode: real	licensed installers are not wired in yet	by design; chapter 4's swap-in runbook is the path

Table 10. Failures and their exits; five rows are field-proven now, and the session row was learned the honest way.

Where the project stands now

Phase 1 is field-verified and closed. Phase 2 is field-verified end to end and closed. Phase 3 is now the same: every runbook step ran green on 28 August 2026, every prediction was reconciled (one missed and explained, one arrived late and better), and an accidental outage proved the forwarding design the way no green run could have: twelve minutes of queue, a daemon restart survived on disk, and 942 late lines delivered with their original timestamps. This guide is edition 2; the only thing still unproven in the phase is the licensed half of the stub contract, which stays a documented swap-in runbook until licenses exist.

Phase 4 is next: the Semaphore UI form that turns these playbooks into a request-driven pipeline, dynamic inventory fed by Terraform state, and the phone_home handshake that closes the loop between a clone and its first contact. Everything Phase 4 orchestrates already exists and already verifies itself; the agent layer landing cleanly is the last piece it was waiting for.

Phase 3

field-verified and closed. Edition 2: every transcript real, one prediction corrected by the field, and an outage that proved the queue better than any planned test could have.